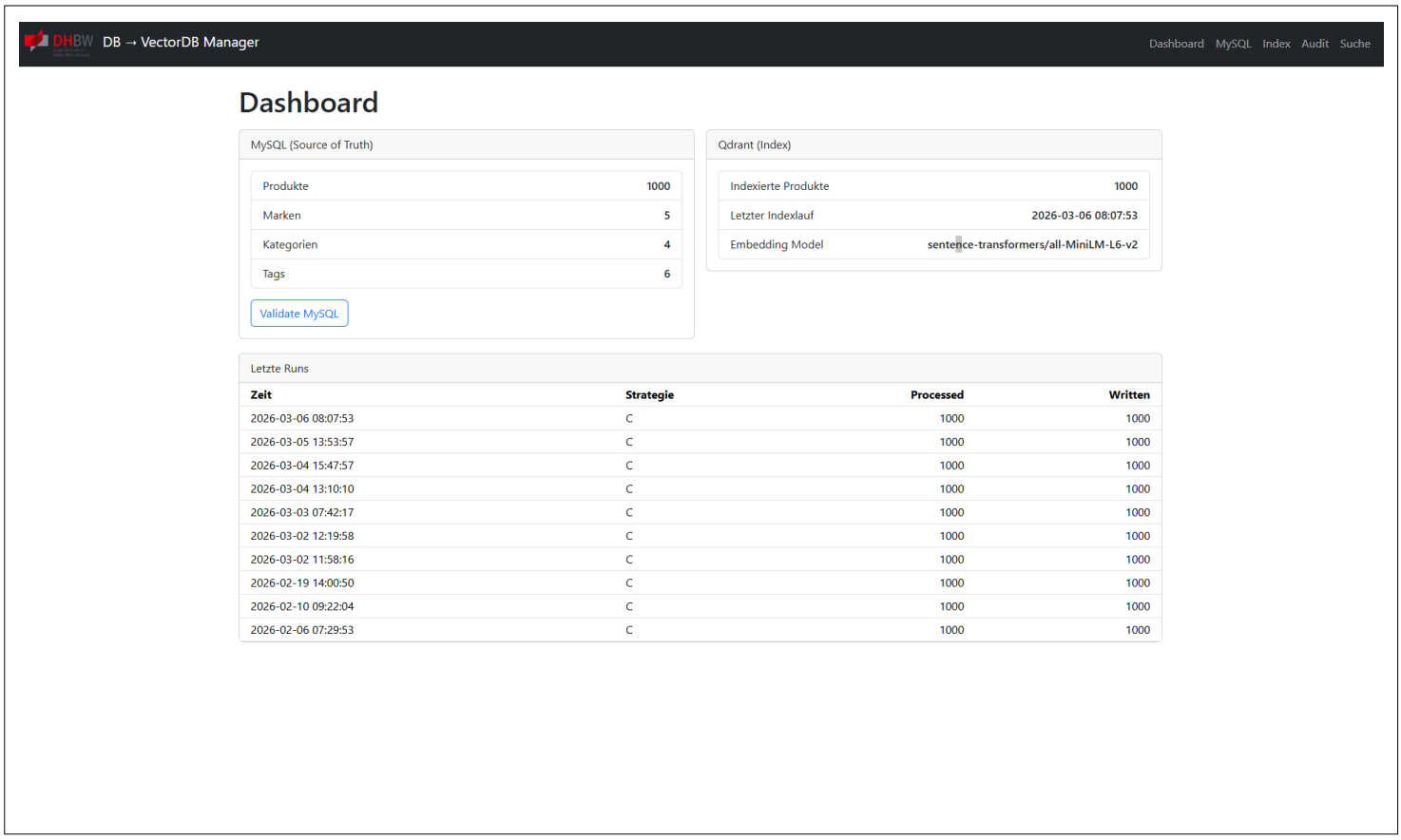
DHBW Stuttgart

Karsten Keßler

Datenbanksysteme TINF24ITA

Projektarbeit ProduktDB

Version 1.1, 27.03.2026



1. Problemstellung und Zielsetzung

In modernen E-Commerce- und Produktinformationssystemen reichen klassische SQL-Abfragen oft nicht aus, um Produkte so zu finden, wie Nutzer sie beschreiben würden. Eine Suche nach „leichter Sommerrucksack für Tageswanderungen" liefert in einer relationalen Datenbank keine Treffer, obwohl passende Produkte vorhanden sind. Vektordatenbanken ermöglichen solche inhaltsbasierten Suchen über natürliche Sprache.

Ziel dieser Projektarbeit ist es, eine relationale Produktdatenbank aufzubauen, diese um eine semantische Suchkomponente (Vektordatenbank) zu ergänzen und beide Ansätze systematisch zu vergleichen: Wann liefert SQL das bessere Ergebnis, wann die semantische Suche?

Die relationale Datenbank bildet dabei stets die Source of Truth.

2. Ausgangslage

- Der Produktdatenbestand wird als CSV-Dateien bereitgestellt und dient als gemeinsame Datenbasis.
- Datenbestand wird nicht inhaltlich verändert
- Fokus auf: Datenmodellierung, SQL-Logik, Architekturverständnis
- Vektorsuche dient ausschließlich der Analyse und Suche, nicht der Persistenz

DHBW Stuttgart

3. Aufgabenstellung

Teil 1 – Relationales Schema & Datenimport
Abgabe: 12.04.2026
Aufgabe A1 (Datenmodell mit Schema- und Import-Skripte)

Teil 2 – Datenbanklogik & semantische Suche
Abgabe: 17.05.2026
Aufgaben A2–A6 (Transaktionen, Trigger, Stored Procedures,
Indizes, Vektor-Datenbank & semantische Suche, lauffähige App, Sourcen)

Sourcen über Github:
https://github.com/karkessler/dhbw-db-2526/

Details auf den folgenden Seiten.

Teil 1 – Relationales Schema & Datenimport
Abgabe: 12.04.2026

A1. Datenmodell
Gegeben ist ein Produktkatalog mit den entsprechenden Entitäten.
Anforderungen:
• vollständiges relationales Schema
• Primär- und Fremdschlüssel
• referenzielle Integrität
• Normalisierung (mindestens 3NF)

Anforderungen an den Import:
• Schema wiederholt aufsetzbar
• saubere Primär- und Fremdschlüssel
• n:m-Zuordnungstabellen
• Verwendung von Transaktionen (START TRANSACTION / COMMIT / ROLLBACK)
• Der Import darf bei Fehlern keine inkonsistenten Daten hinterlassen

Abgabe Teil 1:
• ER-Diagramm (PDF)
• SQL-DDL (schema.sql)
• SQL-Skripte zum Import der CSV-Daten (import.sql)

Teil 2 – Datenbanklogik & semantische Suche
Abgabe: 17.05.2026

A2. Transaktionen
Nutzen und erweitern Sie die im Import-Skript (Teil 1) verwendeten Transaktionskonzepte für fachliche Datenbankoperatiopnen:
• Produkt anlegen inkl. Marke, Kategorie und Tags
• Aktualisieren eines Produkts mit konsistenter Anpassung abhängiger Daten
• Löschen eines Produkts unter Berücksichtigung referenzieller Integrität
• Rollback bei Fehlern (z. B. doppelte Einträge)

Anforderungen:
• START TRANSACTION / COMMIT / ROLLBACK
• sinnvoller Fehlerfall demonstrieren
• keine inkonsistenten Zwischenstände in der Datenbank

Abgabe: SQL-Skript (transaction.sql)

A3. Trigger
Erstellen Sie mindestens einen Trigger, der:
• Änderungen an Produkten protokolliert
• alte und neue Werte speichert
• einen Zeitstempel setzt

Abgabe: SQL-Skript (trigger.sql)

A4. Stored Procedure
Implementieren Sie eine Stored Procedure, z. B.:
• Import neuer Produkte
• automatische Dublettenprüfung
• Validierung von Pflichtfeldern

Abgabe: SQL-Skript (procedure.sql)

A5. Indizes & Baum-Bezug
Erstellen Sie gezielt Indizes, z. B. auf:
• Produktname
• Kategorie
• Marke

Zusätzlich:
• Erklären Sie, warum MySQL hier B-Bäume verwendet
• Analysieren Sie Abfragen mit EXPLAIN

Abgabe: SQL-Skript (index.sql)

A6. Vektor-Datenbank & semantische Suche
• Embedding-Erzeugung für Produktdaten
• Semantische Suche implementieren
• Vergleich zu klassischer SQL-Suche

Optional A7. Implementierung von Graph-DB und LLM-Antworten
• Verbesserungen und Erweiterung der Suchergebnisse.
• Vergleich aller Implementierungen

Abgabe:

• Voll lauffähige App mit entsprechenden Komponenten
• Vergleichsanalyse (Markdown)

4. Kritische Reflexion

Schriftlich zu beantworten (Markdown):
- Grenzen klassischer relationaler Suche
- Grenzen semantischer Suche
- Warum dies keine saubere Datenmodellierung ersetzt
- Warum MySQL die Source of Truth bleibt

5. Optionale Bonusarbeit – Graph-RAG/LLM-Antwort

- Optionale Erweiterung um graphbasierte Kontextanreicherung.
- Kein Nachteil bei Nicht-Umsetzung.

DHBW Stuttgart

6. Bewertung

- ca. 80 % relationaler Teil
- ca. 20 % Vektorteil
- Bonus (max. 4 P.): Graph-RAG/LLM-Erweiterung. Bonuspunkte werden nur angerechnet, sofern die Maximalpunktzahl von 36 nicht erreicht wurde.
- Entwicklungsschritte müssen vollständig nachvollziehbar sein (Commits, Audit-Logs,…), ansonsten Punktabzug

Punkteverteilung (36 P.)

1. Funktionalität (20 P.)
   - Alle Aufgaben erfüllt
   - Fehlerfrei lauffähig
   - Korrektheit und Effizienz des entwickelten Codes bzw. der App
   - Logischer und nachvollziehbarer Aufbau
   - Klar abgegrenzte und sinnvolle Kapitel
   - Theoretischer Hintergrund
   - Nachvollziehbarkeit der Entwicklungsschritte

2. PDF Relationales Model und Modellierung (3 P.)
   - Korrektheit, Vollständigkeit u. Übersichtlichkeit

3. Dokumentation (4 P.) der App und Architektur
   - vollständig als Markdown

4. Kreativität (4 P.)
   - Kreativität in der Herangehensweise und Problemlösung
   - Unkonventionelle und originelle Ansätze

5. Leistungsniveau (5 P.)
   - Vergleich mit den Erwartungen an Informatiker bzgl. modernen Standards
   - Gesamtqualität der Arbeit

7. Präsentation

- ca. 15 Minuten
- Lauffähige App
- Architekturüberblick
- Eine Designentscheidung erläutern
- Lessons Learned

Viel Erfolg!